

Aula 04 - Analista de Dados

UC4: Machine Learning e Data Mining

Tema: Classificação com KNN e Acurácia Básica

Nosso Roteiro de Hoje

- Parte 1: A intuição do algoritmo KNN
- Parte 2: Distância Euclidiana e Escala de Dados
- Parte 3: O Hiperparâmetro K (Overfitting vs Underfitting)
- Parte 4: Hands-on no Python: KNN e a Acurácia

Principais Conceitos de Hoje

Para modelar com KNN, precisamos dominar estes pilares:

- **Algoritmo KNN:** Classifica pontos pela votação de vizinhos mais próximos.
- **Distância Euclidiana:** Métrica geométrica para calcular proximidade.
- **Escala de Dados:** Normalização para dar pesos iguais a todas as colunas.
- **Hiperparâmetro K:** Define o tamanho do grupo de votantes (ruído vs. suavidade).



1. O que é o KNN?

- **K-Nearest Neighbors** (K-Vizinhos Mais Próximos).
- Algoritmo de **Aprendizado Supervisionado** (Classificação).
- **Lógica:** *"Diga-me com quem andas e te direi quem és."*
- Assume que pontos de dados semelhantes existem próximos uns dos outros no espaço geométrico.

Como funciona a classificação?

1. Um novo dado (sem classe) entra no sistema.
2. O KNN calcula a distância entre esse novo ponto e todos os outros pontos já conhecidos (treino).
3. Seleciona-se os K vizinhos com as menores distâncias.
4. **Votação Majoritária:** A classe mais comum entre esses K vizinhos é atribuída ao novo ponto.



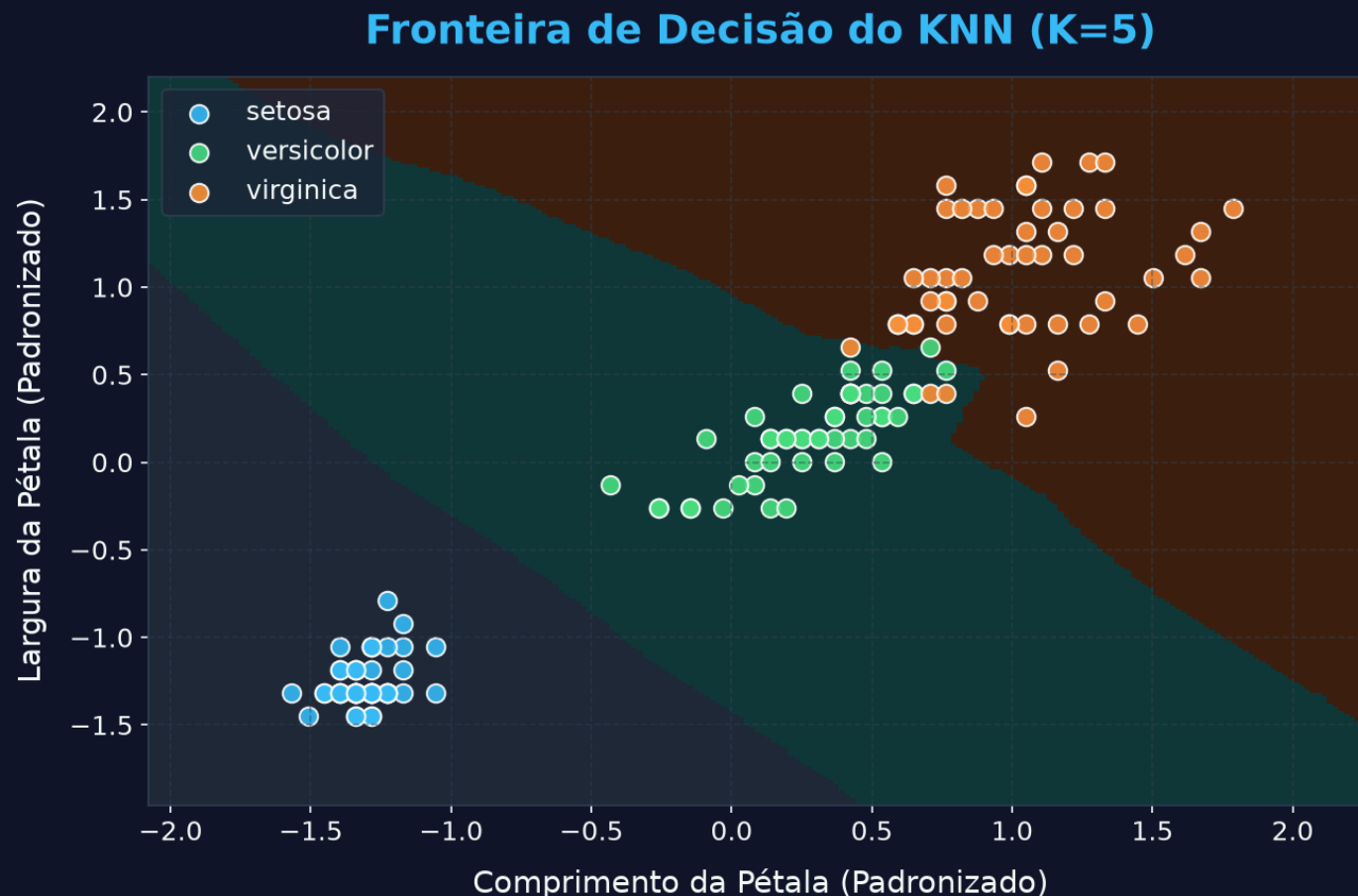
Exemplo Prático de Votação ($K = 3$)

Queremos prever se um cliente de banco vai **Pagar** ou **Dever** a dívida.

- Um cliente novo chega com salário médio.
- Calculamos as distâncias. Os 3 vizinhos mais próximos são:
 - **Vizinho 1:** Deve (distância = 1.2)
 - **Vizinho 2:** Paga (distância = 1.5)
 - **Vizinho 3:** Paga (distância = 1.8)
- **Votos:** 2 "Paga" vs 1 "Deve".
- **Resultado:** O cliente novo é classificado como **Paga!**

🎨 Visão Geral: Fronteiras de Decisão

Abaixo vemos como o KNN divide o espaço de características (Iris) para classificar novos pontos:





2. Distância Euclidiana

Para medir a proximidade entre dois pontos, usamos a fórmula geométrica clássica de distância:

$$d(A, B) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

O algoritmo encontra os vizinhos com as menores distâncias calculadas.

Aplicando a Fórmula na Prática

Imagine dois pontos no plano bidimensional:

- Ponto A (Treino): (Idade = 20, Renda = 5)
- Ponto B (Novo): (Idade = 23, Renda = 9)

$$d(A, B) = \sqrt{(23 - 20)^2 + (9 - 5)^2}$$

$$d(A, B) = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25} = 5$$

→ A distância geométrica entre os pontos é 5.



O Problema da Escala de Dados

O KNN é extremamente sensível à grandeza dos números:

- **Feature Renda:** varia de R\$ 0 a R\$ 1.000.000
- **Feature Idade:** varia de 0 a 100 anos

Se fizermos a diferença de Renda $(100.000 - 5.000)^2 = 9.025.000.000$, a diferença de idade $(40 - 20)^2 = 400$ vira poeira na raiz quadrada. A idade é totalmente ignorada pelo modelo!



Caso Real: O Diagnóstico Médico

Queremos prever se um paciente tem uma Doença da Terceira Idade (Sim/Não).

- **Features:** Idade (0 a 100) e Renda (R\$ 0 a R\$ 1.000.000).
- **Fato:** A doença depende apenas da idade (idosos adoecem, jovens não). Renda é irrelevante.
- **O Erro Sem Escala:** Como a Renda dominará a distância, um jovem rico de 20 anos será agrupado com idosos ricos de 80 anos e receberá um diagnóstico falso de doença!

A Solução: Standard Scaler (Padronização)

Colocamos todas as variáveis sob a mesma escala:

$$Z = \frac{x - \mu}{\sigma}$$

- x : Valor original da célula.
- μ (Média): Média da respectiva coluna.
- σ (Desvio Padrão): Mede a dispersão da coluna (é a raiz quadrada da variância).
- *Como funciona*: Aplica-se essa fórmula coluna por coluna, garantindo pesos iguais a todas as features!



O Diagnóstico Correto (Com Escala)

Após aplicar o `StandardScaler` a ambas as colunas:

- A Renda do jovem e do idoso são mapeadas para a mesma escala (Z).
- A diferença de 60 anos na Idade passa a pesar no cálculo da distância tanto quanto a diferença de Renda.
- O KNN agrupa o jovem de 20 anos com outros jovens e prevê corretamente: Saudável!

⚙️ Parâmetros vs. Hiperparâmetros

Antes de ajustar o K , precisamos entender essa diferença conceitual:

- **Parâmetros:** São os valores que o modelo descobre/aprende sozinho olhando os dados de treino.
Ex: A inclinação (a) e o intercepto (b) na reta de regressão.
- **Hiperparâmetros:** São as configurações que nós definimos manualmente antes do modelo começar a treinar.
Ex: O K no KNN, ou a profundidade (`max_depth`) na Árvore.

3. O papel do Hiperparâmetro K

O valor de K (número de vizinhos votantes) altera a sensibilidade do modelo:

- K muito pequeno ($K = 1$): Muito sensível a ruídos. Causa **Overfitting** (decoreba e erro em dados novos).
- K muito grande ($K = 51$): Fronteira muito rígida, ignora padrões locais. Causa **Underfitting** (simplificação).
- *Regra Prática*: Use sempre valores ímpares para evitar empates!



Analogia: A Prova de Matemática

- **Overfitting** ($K = 1$ - Decoreba): O aluno decora as respostas exatas dos exercícios do livro. Se a prova mudar um número da questão, ele erra porque só decorou.
- **Underfitting** ($K = 51$ - Preguiça): O aluno decora apenas uma regra super simples (ex: "somar tudo"). Ele erra a prova toda por simplificar demais.
- **Generalização** ($K = 5$ - Equilíbrio): O aluno entende a lógica das equações. Ele resolve problemas novos com números inéditos na prova.



Como Escolhemos o K Ideal?

- **Sem fórmulas diretas:** Nós testamos vários valores de K (1, 3, 5, 9...) e plotamos a acurácia de cada um em um gráfico.
- O K com a melhor acurácia na base de teste é o escolhido.
- **Na prática do mercado:** Automatizamos essa busca rodando uma técnica de tentativa e erro programada chamada **Grid Search** (Busca em Grade).

4. Acurácia (Accuracy)

Mede a taxa global de acertos do nosso classificador:

$$\text{Acurácia} = \frac{\text{Previsões Corretas}}{\text{Total de Previsões}}$$

- **Exemplo:** Se de 100 flores de teste o KNN classifica 96 corretamente, a Acurácia é de 96%.



KNN com Scikit-Learn (Resumo de Código)

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

# 1. Padronizar as features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 2. Instanciar e treinar o KNN com K=5
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_scaled, y_train)

# 3. Prever e calcular acurácia
previsoes = knn.predict(X_test_scaled)
acuracia = accuracy_score(y_test, previsoes)
```



Desafio Prático da Aula 04

1. Carregue o dataset Iris e faça o split 70/30.
2. Normalize os dados de treino e teste usando o `StandardScaler`.
3. Treine o classificador KNN para $K = 1, 5$ e 21 .
4. Compare as acurácias obtidas no conjunto de teste. Qual K obteve a melhor performance?
5. Remova a etapa de normalização e treine o KNN novamente. O que acontece com a acurácia?

Conclusão

Hoje aprendemos a classificar pontos com base em seus vizinhos e a importância de normalizar dados de distância!

Próxima Aula: Matriz de Confusão, Precisão, Recall e F1-Score (Avaliando modelos como profissionais).